

Unit II: Advanced Data Types and Decision Control Statements

Section 1: Advanced Data Types in Python

Python provides **powerful built-in data structures** to store, manage, and manipulate collections of data efficiently.

1.1 Lists

- **Definition:** Ordered, mutable (can change), allows duplicates.
- **Syntax:**

```
fruits = ["apple", "banana", "cherry"]
```

- **Key Features:**
 - Ordered → Maintains insertion order
 - Mutable → Elements can be modified
 - Allows duplicate values

Common Operations:

Operation	Syntax	Example
Access element	<code>list[index]</code>	<code>fruits[0] → "apple"</code>
Modify element	<code>list[index] = value</code>	<code>fruits[1] = "orange"</code>
Append element	<code>list.append(value)</code>	<code>fruits.append("mango")</code>
Insert at position	<code>list.insert(index, value)</code>	<code>fruits.insert(1, "kiwi")</code>
Remove element	<code>list.remove(value)</code>	<code>fruits.remove("apple")</code>
Length of list	<code>len(list)</code>	<code>len(fruits) → 3</code>

Example:

```
numbers = [1, 2, 3, 4]
numbers.append(5)
print(numbers)  # Output: [1, 2, 3, 4, 5]
```

1.2 Tuples

- **Definition:** Ordered, immutable (cannot change after creation), allows duplicates.

- **Syntax:**

```
coords = (10, 20, 30)
```

- **Key Features:**

- Immutable → Cannot modify elements
- Ordered → Maintains order
- Allows duplicates

Example:

```
point = (5, 10)
print(point[0]) # Output: 5
```

- **Use Case:** Tuples are ideal for **fixed data**, such as coordinates (x , y) or RGB colors (255, 0, 0).
-

1.3 Sets

- **Definition:** Unordered, mutable, **no duplicate elements**.
- **Syntax:**

```
unique_numbers = {1, 2, 3, 3, 4} # Duplicate 3 removed
```

- **Key Features:**

- Unordered → No guaranteed order
- Mutable → Can add/remove elements
- No duplicates → Automatically removes repeated values

Set Operations:

Operation	Example	Output
Add element	<code>s.add(5)</code>	{1, 2, 3, 4, 5}
Remove element	<code>s.remove(3)</code>	{1, 2, 4, 5}
Union	<code>a b</code>	<code>a b</code>
Intersection	<code>a & b</code>	{2, 3}
Difference	<code>a - b</code>	{1}

Example:

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a & b) # Output: {2, 3} (Intersection)
```

1.4 Dictionaries

- **Definition:** Unordered collection of **key-value pairs**, mutable, keys must be unique.
- **Syntax:**

```
student = {"name": "Alice", "age": 20, "grade": "A"}
```

- **Key Features:**
 - Keys → Must be unique and immutable
 - Values → Can be any data type
 - Mutable → Add or modify key-value pairs

Common Operations:

Operation	Syntax	Example
Access value	dict[key]	student["name"] → "Alice"
Modify value	dict[key] = value	student["age"] = 21
Add new pair	dict[new_key] = value	student["city"] = "Pune"
Delete pair	del dict[key]	del student["grade"]
Keys & Values	dict.keys(), dict.values()	dict.keys() → ["name", "age", "city"]

Example:

```
person = {"name": "Bob", "city": "Mumbai"}
person["age"] = 25
print(person)  # {'name': 'Bob', 'city': 'Mumbai', 'age': 25}
```

Section 2: Decision Control Statements

Decision control statements allow **programs to execute code based on conditions**.

2.1 if Statement

- Executes **block only if condition is true**.

```
x = 10
if x > 5:
    print("x is greater than 5")
```

2.2 if-else Statement

- Executes **one block if true**, another if false.

```
x = 3
if x % 2 == 0:
    print("Even")
else:
    print("Odd")
```

2.3 Nested if

- An if statement inside another if.

```
x = 10
if x > 0:
    if x < 20:
        print("x is between 1 and 19")
```

2.4 if-elif-else

- Used for multiple exclusive conditions.

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Section 3: Loop / Iterative Statements

Loops allow **repeated execution** of a block of code.

3.1 while Loop

- Repeats as long as a condition is true.

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

3.2 for Loop

- Iterates **over a sequence** (list, tuple, string, range).

```
for i in range(1, 6):  
    print(i)
```

3.3 Nested Loops

- A **loop inside a loop**, useful for multi-dimensional data.

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, j)
```

3.4 Loop Control Statements

Statement	Description	Example
break	Exits loop immediately	for i in range(5): if i==3: break
continue	Skips current iteration	for i in range(5): if i==3: continue
pass	Placeholder for empty block	for i in range(5): pass
else with loop	Executes if loop completes normally	for i in range(3): print(i) else: print("Done")

3.5 Choosing the Appropriate Loop

- **for loop:** Use when **number of iterations is known**.
 - **while loop:** Use when **iteration depends on a condition**.
 - **Nested loops:** For **multi-dimensional structures** like matrices.
-

Key Takeaways for Students

- Lists, tuples, sets, and dictionaries are **essential for data storage and manipulation**.
- Decision control statements (**if, if-else, if-elif-else**) **control the flow of a program**.
- Loops (**for, while**) **enable repetition**, while **break/continue/pass** provide **fine control**.